AI ASS 12.4

Name:M.Geethika

H.T.NO:2303A52276

Batch:43

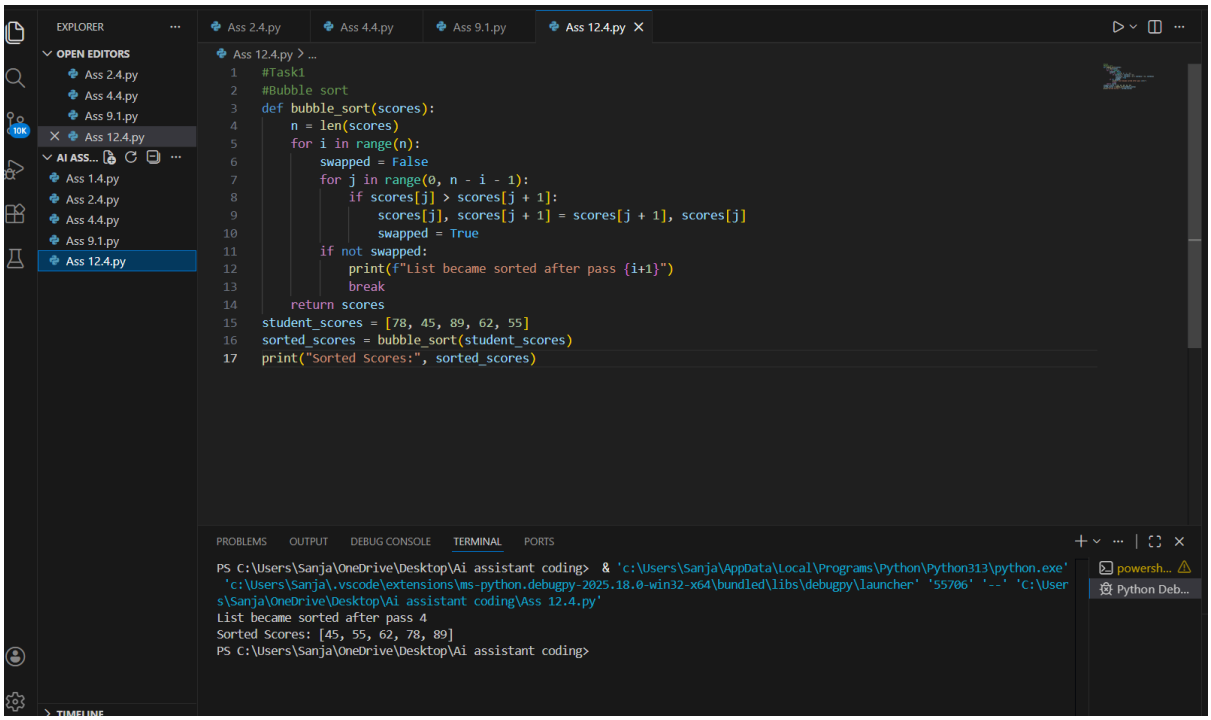
Task1:

Write a Python program to implement Bubble Sort for sorting student exam scores.

Add inline comments explaining comparisons, swaps, and passes.

Include early termination using a swapped flag.

Provide sample input/output and briefly explain best, average, and worst-case time complexity.



Observation:

Bubble Sort was implemented to sort student scores after internal assessments.

The algorithm shows O(n) time complexity in the best case when the list is already sorted due to early termination.
In the average case, when scores are randomly arranged, the time complexity is $O(n^2)$.
In the worst case, when scores are in reverse order, it also takes $O(n^2)$ time due to maximum comparisons and swaps.
Thus, Bubble Sort is suitable for small datasets but not efficient for large datasets.

Task2:

I have a nearly sorted list of student roll numbers (attendance records).

1. Implement Bubble Sort in Python.

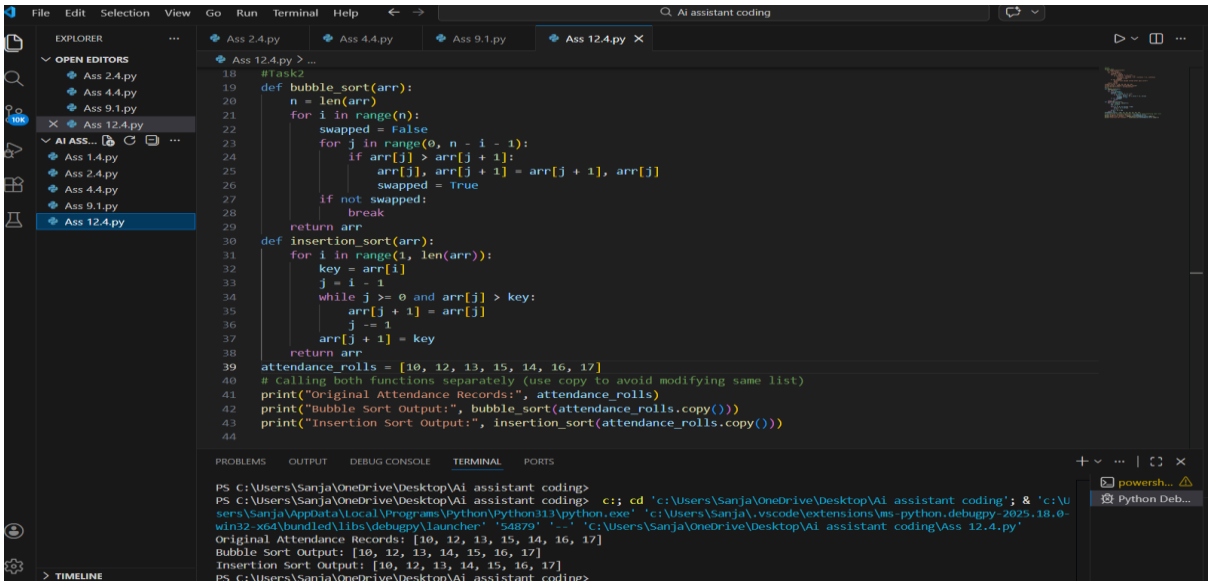
2. Suggest a more suitable sorting algorithm for nearly sorted data.

3. Implement Insertion Sort in Python.

4. Explain why Insertion Sort performs better on nearly sorted datasets.

5. Compare their behavior on a nearly sorted input example.

6. Include time complexity analysis.



Observation:

1.The attendance roll numbers were nearly sorted with only a few misplaced elements.

2.Bubble Sort performed multiple comparisons even when only small corrections were needed.

3.Insertion Sort shifted only the misplaced elements to their correct positions.

4.For nearly sorted data, Insertion Sort behaved close to **O(n)** time complexity.

5. Bubble Sort still required more passes, making it comparatively slower.

6.Therefore, Insertion Sort is more efficient and suitable for partially sorted attendance records.

Task3:

Building a student information portal.

1. Implement Linear Search for unsorted student roll numbers.

2. Implement Binary Search for sorted roll numbers.

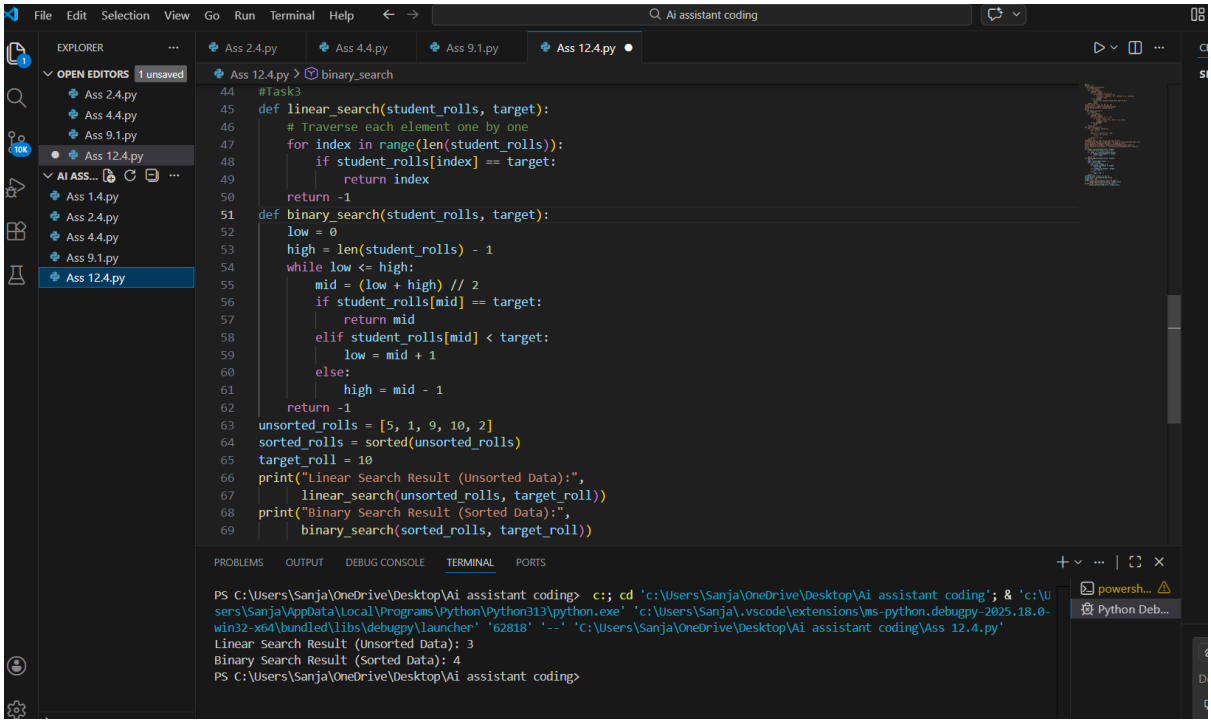
3. Add proper Python docstrings explaining parameters and return values.

4. Explain when Binary Search can be used.

5. Compare time complexity and performance differences.

6. Provide sample input and output.

7. Add a short observation comparing results on sorted vs unsorted data.



Observation:

Linear Search checks each element one by one, so its time complexity is **O(n)** in average and worst cases. It works on both sorted and unsorted lists.

Binary Search divides the sorted list into halves repeatedly, so its time complexity is **O(log n)**. It works only on sorted data.

For unsorted lists, Linear Search is used.
For large sorted lists, Binary Search is faster and more efficient.

Task4:

work with large datasets that may be random, already sorted, or reverse sorted.

1. Complete recursive implementations of Quick Sort and Merge Sort in Python.

2. Add proper docstrings explaining parameters and return values.

3. Explain how recursion works in both algorithms.

4. Test both algorithms on:

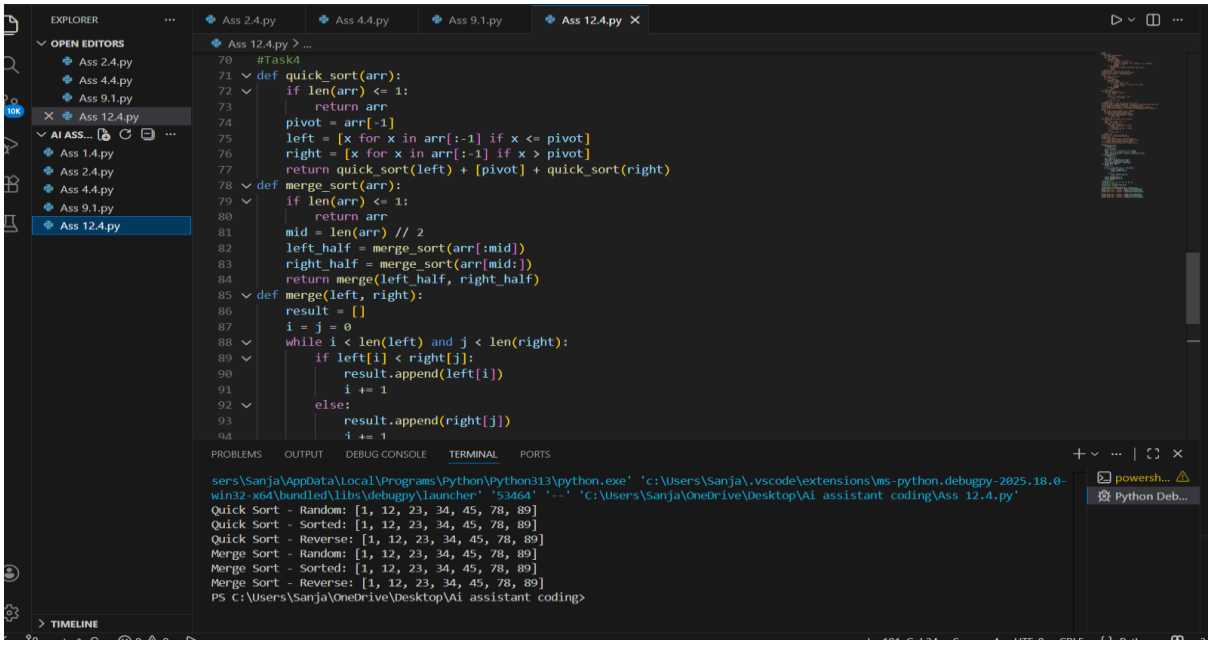
- Random data

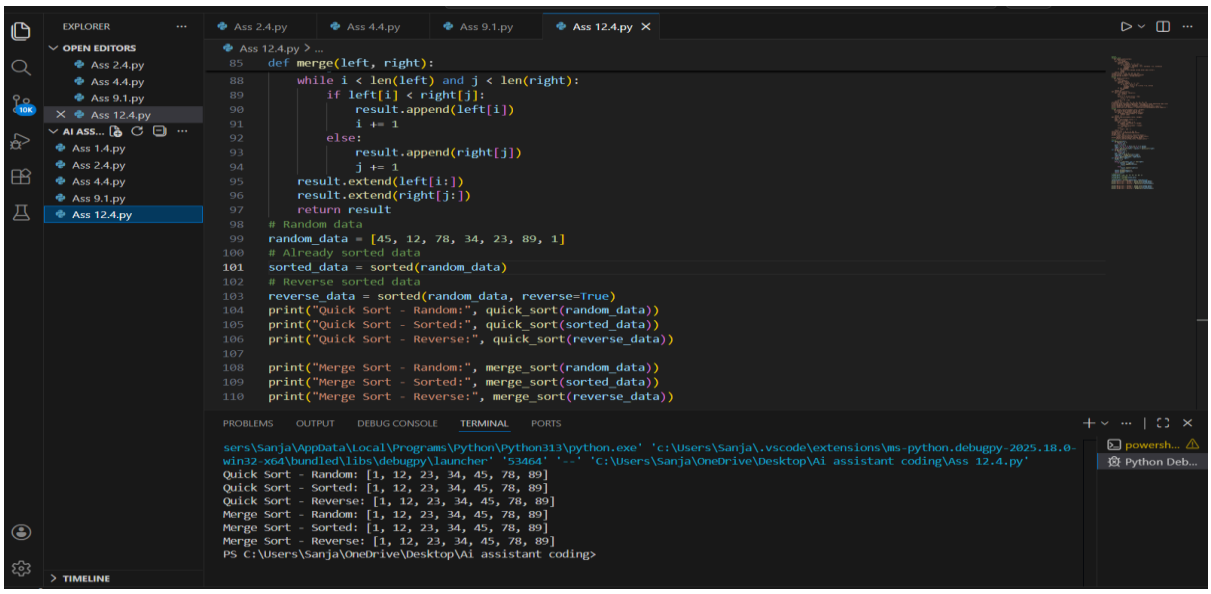
- Sorted data

- Reverse sorted data

5. Compare best, average, and worst-case time complexity.

6. Explain practical scenarios where one is preferred over the other.





Observation:

1.Quick Sort has best and average time complexity of O(n log n).

2.Quick Sort worst case is $O(n^2)$ when pivot selection is poor.

3.Merge Sort has O(n log n) time complexity in best, average, and worst cases.

4.Quick Sort uses less memory (in-place), while Merge Sort requires extra memory.

5.Quick Sort is usually faster for random datasets.

6.Merge Sort is preferred when guaranteed performance and stable sorting are required.

Task5:

Building a data validation module to detect duplicate user IDs.

1. Write a brute-force duplicate detection algorithm using nested loops.

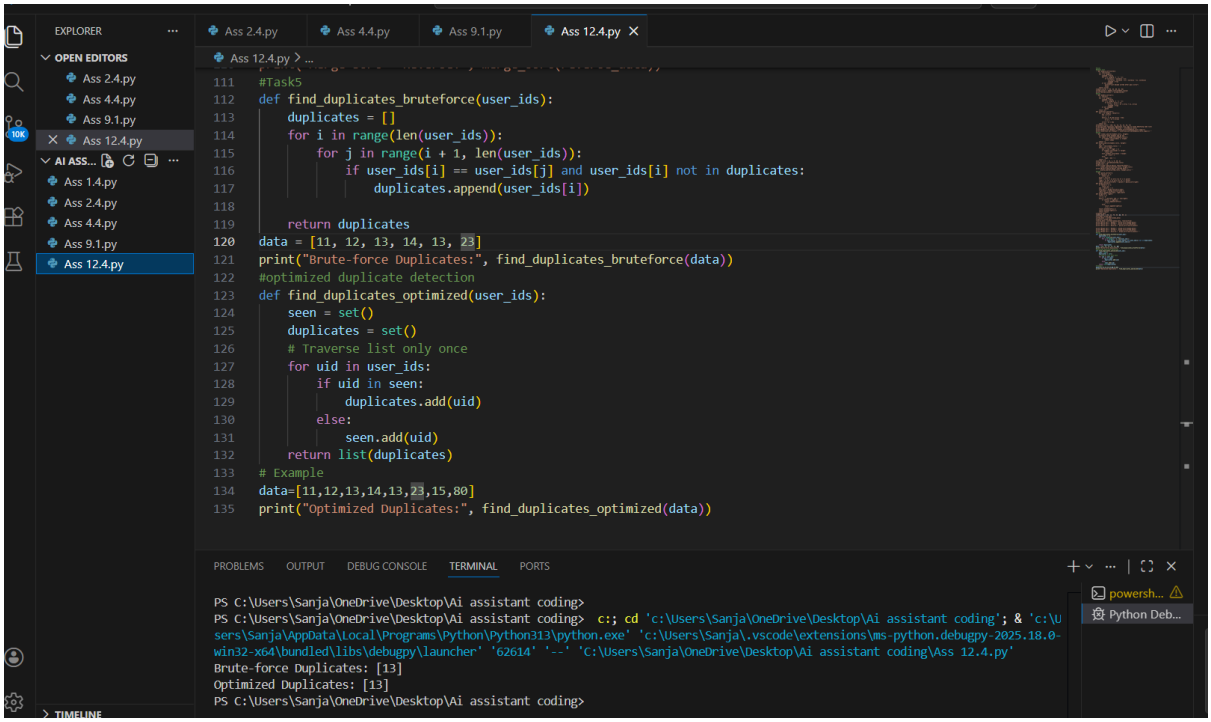
2. Analyze its time complexity.

3. Suggest an optimized approach using a set or dictionary.

4. Rewrite the algorithm with improved efficiency.

5. Compare performance conceptually for large datasets.

6. Keep explanation simple and clear.



Observation:

1.The brute-force algorithm uses nested loops to compare every element with all others, resulting in $O(n^2)$ time complexity.

2.It performs a large number of unnecessary comparisons, especially for large datasets.

3.The optimized algorithm uses a set (or dictionary) to store already seen elements.

4.Since set lookup takes O(1) time, the entire list is processed in a single loop, giving O(n) time complexity.

5.The performance improved because repeated comparisons were replaced with fast lookups.

6.Therefore, the optimized approach is much faster and more suitable for large datasets.